kauno kolegija | Faculty of Technologies

.NET frontend technologies

Razor Pages, Blazor Server and Blazor WebAssembly on the same SimpleStudent API

Group PSe-5

Autumn 2026

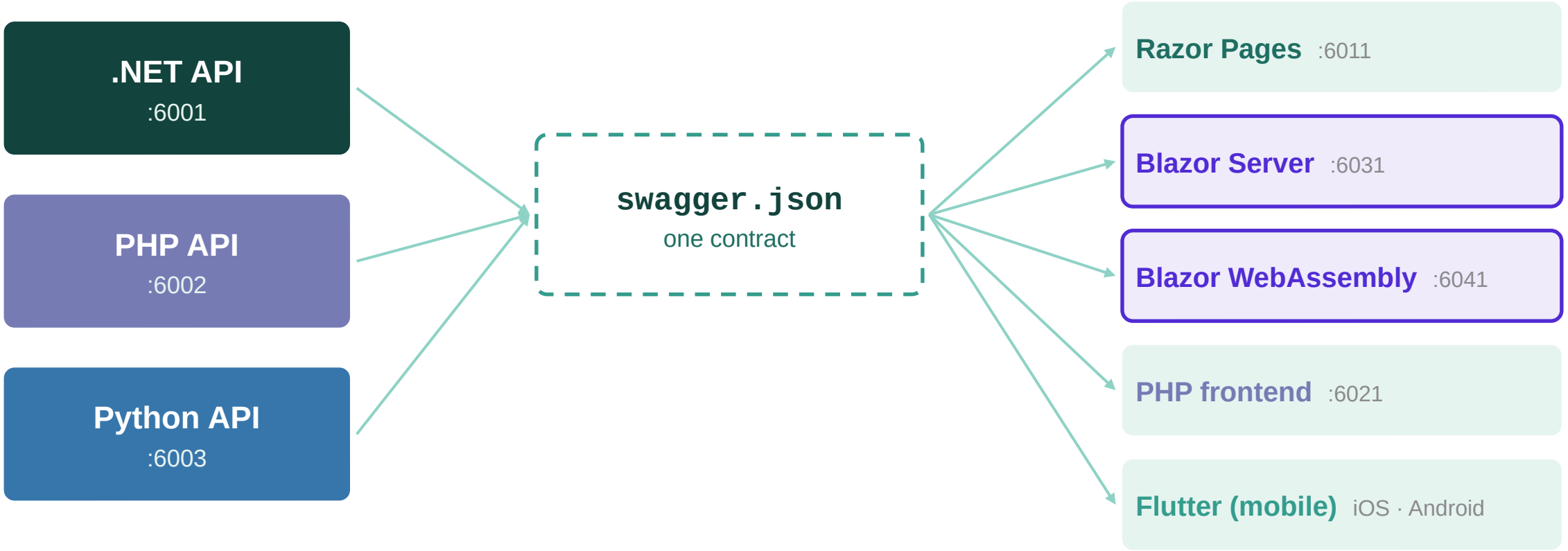
Lecture + practice

Lecturer: Tadas Bulis

WHERE IT FITS

One API contract, many clients

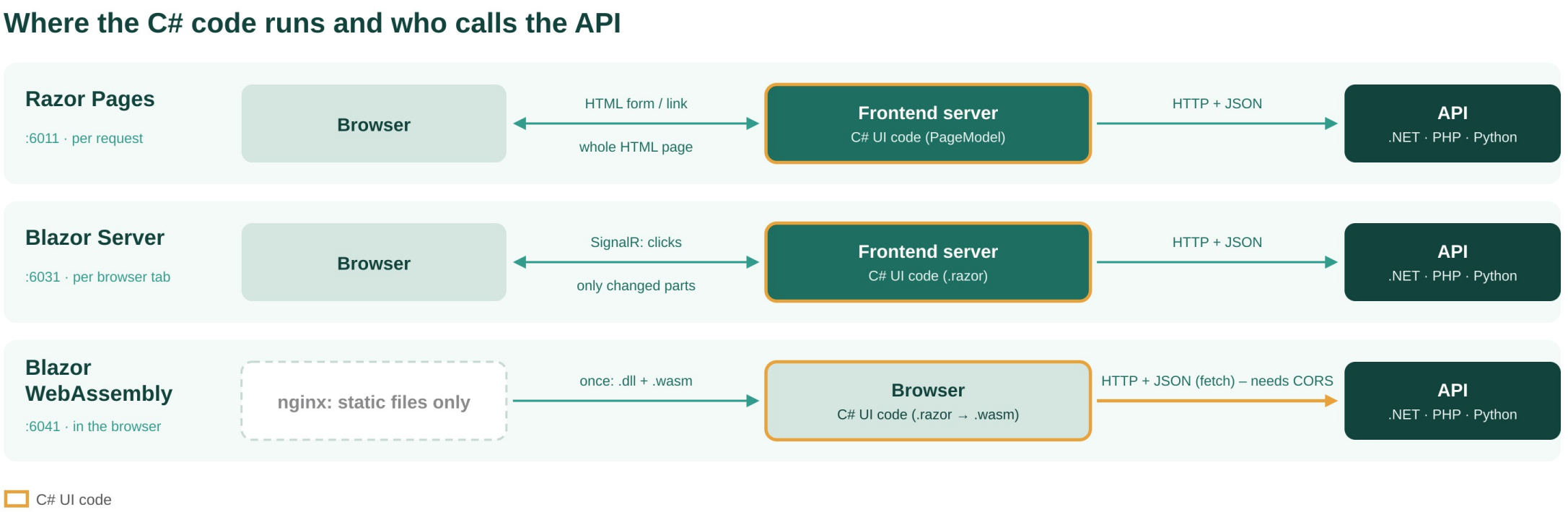
Three API implementations and five clients, all built on the same swagger.json



Purple frame = the new Blazor clients

Where the C# code runs and who calls the API

Same API, same generated client, a different place for the UI code



Razor Pages: one page per request

SimpleStudentFrontend · :6011 · .cshtml + PageModel

1. **GET /Students** – the browser asks for a page
2. **OnGetAsync()** on the server calls the API via IStudentApiClient
3. **Whole HTML** is rendered and sent; the PageModel is discarded
4. **Action = new request**: POST → RedirectToPage() → GET

- The simplest and most robust model: works without WebSocket, great for SEO
- Interactivity without reloads needs JavaScript

Blazor Server: components live on the server

SimpleStudentBlazorServer · :6031 · .razor components, render mode Interactive Server

1. **blazor.web.js** opens a SignalR (WebSocket) connection
2. **Circuit**: the server keeps component state for every tab
3. **A click** travels over SignalR; the server runs the C# method
4. **Only changes** go back to the browser; the server calls the API

- Interactive without JavaScript, fast first load, code and keys stay on the server
- Every user = memory on the server; when the network drops the UI stops (“reconnect”)

Blazor WebAssembly: the app runs in the browser

SimpleStudentBlazorWasm · :6041 · the same .razor components + nginx

1. **nginx** serves index.html, .wasm and .dll – no .NET on the server
2. **.NET runtime** starts in the browser (WebAssemblyHostBuilder)
3. **Components** run in the browser – clicks need no server
4. **The browser calls the API** via fetch() → needs CORS and a public address

- Static hosting (CDN), works offline, load moves to the client
- Larger first download; every .dll and wwwroot/appsettings.json is public

COMPARISON

Three technologies in one table

	Razor Pages	Blazor Server	Blazor WebAssembly
Where the UI C# code runs	Server, per request	Server, per tab	Browser
Page update	Whole page	Only changed parts	Only changed parts
Who calls the API	Server	Server	Browser
CORS on the API	Not needed	Not needed	Required
First load	Fast	Fast	Slower (a few MB)
Server load	Only during a request	Memory for every user	Minimal (static files)
When the network drops	Next click works	UI stops, “reconnect”	UI works, API does not
Secrets	Can stay on the server	Can stay on the server	Not allowed – all public
Hosting	ASP.NET Core	ASP.NET Core	nginx / any static host
Our project	:6011	:6031	:6041

SAME TASK

Loading the list of students

Razor Pages vs Blazor (Server and WASM code is identical)

Razor Pages — Pages/Students.cshtml.cs

```
public class StudentsModel : PageModel
{
  public List<Student> Students { get; set; }

  public async Task OnGetAsync()
  {
    // GET /api/students
    Students = (await _api.GetStudentsAsync())
                    .ToList();
  }
}
```

Blazor — Pages/Students.razor

```
@page "/students"
@inject ApiConnection Api

<input @bind="_search"
       @bind:event="oninput" />
@foreach (var s in Filtered) { <tr>…</tr> }

@code {
  List<Student> _students = new();
  protected override async Task
      OnInitializedAsync() => _students =
      (await Api.Client.GetStudentsAsync()).ToList();
}
```

Razor Pages: data is loaded during the request, the template renders it once. **Blazor:** when a field changes, Blazor re-renders that part of the HTML.

SAME TASK

Deleting a student

Form and redirect vs one C# method

Razor Pages — Students.cshtml(.cs)

```
<form method="post"
      asp-page-handler="Delete">
  <input type="hidden" name="id"
         value="@s.Id" />
  <button type="submit">Delete</button>
</form>

public async Task<IActionResult>
    OnPostDeleteAsync(int id)
{
  await _api.DeleteStudentAsync(id);
  return RedirectToPage(); // whole page
}
```

Blazor — Pages/Students.razor

```
<button @onclick="() => DeleteAsync(s.Id)">
  Yes
</button>

@code {
  async Task DeleteAsync(int id)
  {
    await Api.Client.DeleteStudentAsync(id);
    _students.RemoveAll(s => s.Id == id);
    // only this row disappears
  }
}
```

Razor Pages: POST → 302 → GET, the whole page reloads. **Blazor:** one method, one table row changes in the browser.

Program.cs: Blazor Server vs WebAssembly

The page components are the same – only where they start differs

SimpleStudentBlazorServer/Program.cs

```
// runs ON THE SERVER
builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();
builder.Services.AddScoped<ApiConnection>();
...
app.MapRazorComponents<App>()
   .AddInteractiveServerRenderMode();

// Components/App.razor
<Routes @rendermode="new
  InteractiveServerRenderMode(prerender: false)"/>
```

SimpleStudentBlazorWasm/Program.cs

```
// runs IN THE BROWSER
var builder = WebAssemblyHostBuilder
                  .CreateDefault(args);
builder.RootComponents.Add<App>("#app");
builder.Services.AddScoped<ApiConnection>();
await builder.Build().RunAsync();

// Pages/*.razor, Layout/*, Shared/*
// – the SAME files in both projects
```

Razor Pages registers AddRazorPages() + MapRazorPages(). **prerender: false** – so the API is not called twice.

Validation rules come from the contract

Shared/StudentForm.razor – one form for create and edit



Blazor — Shared/StudentForm.razor

```
<EditForm EditContext="_editContext"
          OnValidSubmit="SubmitAsync" novalidate>
  <DataAnnotationsValidator />
  <InputText @bind-Value="Model.FirstName" />
  <ValidationMessage For="() => Model.FirstName"/>
</EditForm>
```

Pitfall: type="email"

The browser's HTML5 validation silently blocks the form with the value “x”: neither Blazor nor API errors appear.

Fix: novalidate on EditForm. We found this bug while testing the example.

Server address or browser address?

The most important practical difference between Blazor Server and WebAssembly

Razor Pages and Blazor Server

The server calls the API inside the Docker network (simplestudentapi:8080). No CORS needed, keys can stay on the server.

Blazor WebAssembly

The browser calls the API at an address it can see (localhost:6001). CORS is required, all of wwwroot is public.

docker-compose.yml (Blazor Server)

```
Backends__0__BaseUrl:
  http://simplestudentapi:8080
```

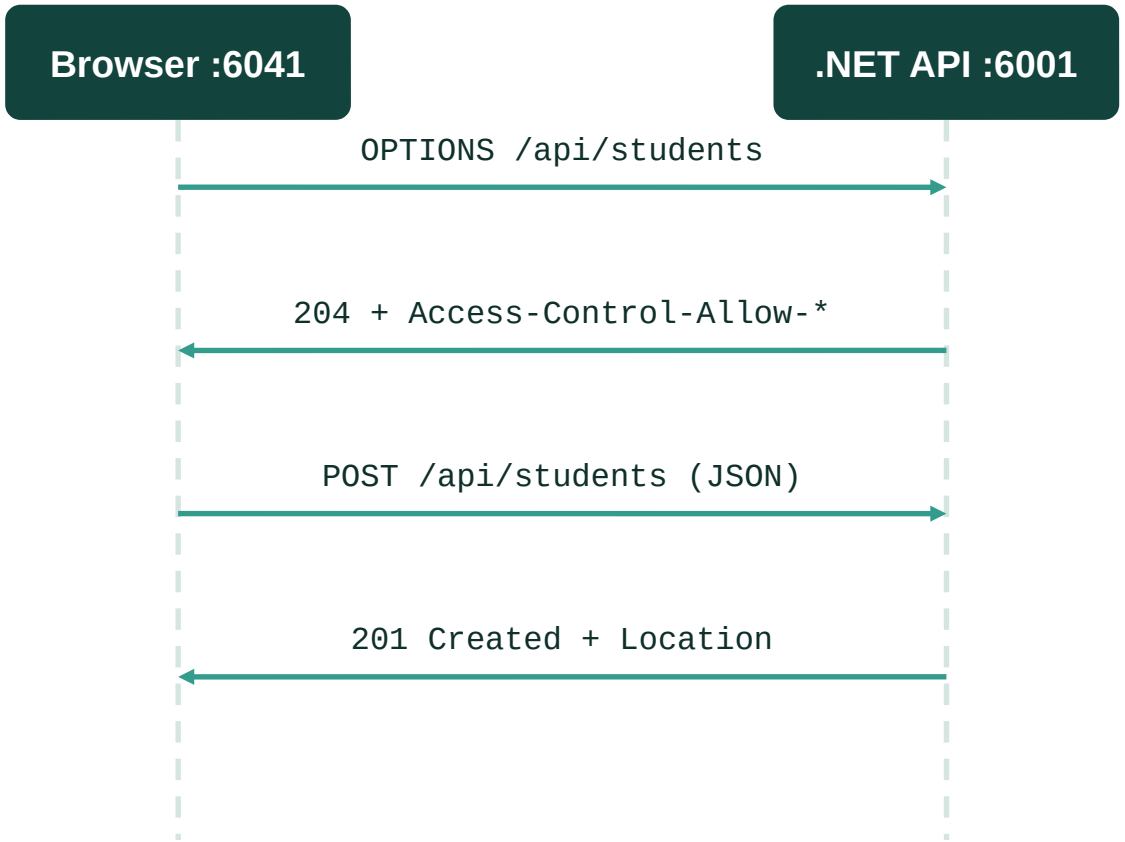
wwwroot/appsettings.json (WebAssembly)

```
{ "Name": ".NET",
  "Url": "http://localhost:6001" }
```

F12 → Network: in Razor Pages and Blazor Server you see no requests to :6001, in WebAssembly you see every one.

Why only WebAssembly needs CORS

The browser asks the API for permission before sending JSON from another address



.NET API — SimpleStudentApi/Program.cs

```
builder.Services.AddCors(options =>
  options.AddDefaultPolicy(policy => policy
    .AllowAnyOrigin()
    .AllowAnyHeader()
    .AllowAnyMethod()
    .WithExposedHeaders("Location")));
...
app.UseCors();
```

Why only WebAssembly?

CORS is a browser rule; servers do not enforce it. The PHP and Python APIs already had CORS, the .NET API was extended.

CHOOSING

When to choose which

The users, the network and the hosting decide – not the API

Razor Pages

Choose when…

- content site, forms
- SEO matters
- simplest solution needed

Be careful if…

- lots of interactivity without reloads

Blazor Server

Choose when…

- internal business app
- lots of interactivity
- API and DB in a private network

Be careful if…

- many users at the same time
- unstable network

Blazor WASM

Choose when…

- static hosting, CDN
- offline use (PWA)
- load moves to the client

Be careful if…

- large first download
- API must be public + CORS

COMMON PROBLEMS

What you will meet in practice

We hit all four while building the examples

BLAZOR SERVER

The API is called twice

Prerendering renders the page twice. Ours: InteractiveServerRenderMode(prerender: false).

BLAZOR SERVER

“Attempting to reconnect…”

The SignalR connection dropped. The page reconnects itself, but the circuit state (e.g. the chosen API) is lost.

BLAZOR WASM

“TypeError: Failed to fetch”

The API is not running or does not allow CORS. F12 → Console shows the missing header.

BLAZOR WASM

simplestudentapi:8080 fails

That name exists only inside the Docker network. The browser needs localhost:6001.

DEMO

Running and demo

From the folder student-docker-lab-plus-mssql

Terminal

```
docker-compose -f docker-compose.infra.yml up -d
docker-compose up --build
```

Demo steps

1. Open :6031 and :6041 side by side
1. Switch the API to Python – no page reload
1. F12 → Network and delete a student in both
1. docker stop simplestudentblazorserver – watch “reconnect”

:6011	Razor Pages
:6031	Blazor Server
:6041	Blazor WebAssembly
:6021	PHP frontend
:6001 · 6002 · 6003	.NET · PHP · Python API

Tasks for students

Described in detail in DotNet-Frontend-Comparison_EN

1. F12 → Network: count the requests when deleting a student in **all three** frontends

2. Stop the Blazor Server, then the WASM container. **How does** the behaviour differ?

3. Remove `app.UseCors()` from the .NET API. Why do only the server frontends work?

4. Add **sorting by last name** to the Blazor list without a new API request

5. Build a **Faculty** page in the Razor Pages frontend and compare the amount of code

6. Save the email “x” via the PHP API. **Where did** the error message come from?

One API, three ways to show it

Only where the C# code runs and who sends the HTTP request differ

Razor Pages
server, whole page per request

Blazor Server
server, only changes over SignalR

Blazor WebAssembly
browser, calls the API directly + CORS

Lecturer: Tadas Bulis